LAB 7 – FUZZ TESTING

Input Validation Bugs in the Highway Toll Calculator

Name : Anand

SRN : PES1UG24AM341

Section : 5F

Date : 12/11/2026

Project	Highway Toll Calculator
Application	highway_toll.py
Test file	test_highway_toll.py
Testing tools	Hypothesis and pytest
Python version	3.11.9

Objective

Use property-based fuzz testing with Hypothesis to generate unexpected inputs for the three functions in the Highway Toll Calculator, identify input-validation failures, harden the application, and rerun the same tests to verify the fixes.

Functions tested

Function	Purpose
parse_plate()	Parse and validate a vehicle registration plate.
trip_cost()	Calculate trip cost from a trips × rate expression.
axle_class()	Classify a vehicle based on its axle count.

Screenshot 1 – Original Application Output

The original application was executed with valid sample inputs: KA-05-AB-1234, 3x150, and 3. The functions returned the expected sample results.

```
PS C:\Users\anand\OneDrive\Desktop\git\SE_LAB_PES1UG24AM341\SE_Lab7\lab7> python highway_toll.py
==== Running parse_plate ====
Enter a plate (e.g. KA-05-AB-1234): KA-05-AB-1234
State, District and Number: ('KA', 5, 1234)

==== Running trip_cost ====
Enter trips x rate (e.g. 3x150): 3x150
Trip Cost: 450

==== Running axle_class ====
Enter an axle count (e.g. 3): 3
Vehicle Class: truck
```

Screenshot 2 – Hypothesis Fuzz Testing Failures

Hypothesis generated unexpected inputs and all three no-crash tests failed against the original implementation. Examples included empty strings and None values. The detailed test log identified failures such as malformed plate splitting, invalid integer conversion, and attempts to call string methods on None.

```
PS C:\Users\anand\OneDrive\Desktop\git\SE_LAB_PES1UG24AM341\SE_Lab7\lab7> python -m pytest test_highway_toll.py -v
============================= test session starts ==============================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\anand\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\anand\OneDrive\Desktop\git\SE_LAB_PES1UG24AM341\SE_Lab7\lab7
plugins: anyio-4.14.2, hypothesis-6.170.15, typeguard-4.6.0
collected 3 items

test_highway_toll.py::test_parse_plate_no_crash FAILED                   [ 33%]
test_highway_toll.py::test_trip_cost_no_crash FAILED                     [ 66%]
test_highway_toll.py::test_axle_class_no_crash FAILED                    [100%]
```

Observed issues: parse_plate() failed on malformed or None input; trip_cost() failed on missing/invalid separators or None; axle_class() failed on empty or non-numeric input and None.

Screenshot 3 – Fixed Application Output

The application was updated with input type checks, format validation, numeric validation, and defensive handling of invalid values. Valid sample inputs continue to produce the expected results.

```
PS C:\Users\anand\OneDrive\Desktop\git\SE_LAB_PES1UG24AM341\SE_Lab7\lab7> python highway_toll.py
==== Running parse_plate ====
Enter a plate (e.g. KA-05-AB-1234): KA-05-AB-1234
State, District and Number: ('KA', 5, 1234)

==== Running trip_cost ====
Enter trips x rate (e.g. 3x150): 3x150
Trip Cost: 450

==== Running axle_class ====
Enter an axle count (e.g. 3): 3
Vehicle Class: truck
```

Screenshot 4 – Final Hypothesis Test Results

After updating highway_toll.py, the same Hypothesis tests were executed again. All three tests passed, confirming that the tested unexpected inputs no longer caused unhandled exceptions.

Test	Final result
test_parse_plate_no_crash	**PASSED**
test_trip_cost_no_crash	**PASSED**
test_axle_class_no_crash	**PASSED**
Overall	**3 passed**

